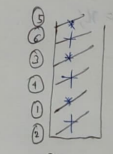
Infix to Postfix Conversion using Stack

1. Read the infix from left to right
2. Initialize an empty stack.
3. While (more input):
   (a) if '(' read : push onto stack
   (b) if operand is read : ~~push onto stack~~ add to output
   (c) if ')' read: (i) pop the stack and add to output until '(' is encountered
   (ii) pop the '(' and discard it
   (iii) discard the ')'.
   (d) if operator is read:
   (i) while (precedence (operator) < precedence (top of stack)):
   pop the stack and add to the output
   (ii) push operator onto stack
4. Pop the stack until stack is empty and add each element to output.

* Never push higher (lower) precedence operator over higher precedence operator

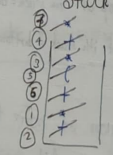
e.g.- a+b*c+(d*e+f)*g

O/P: abc*+de*+fg*+



a+b*c+(d*e+f)*g

O/P: abc*+de*f+g*+



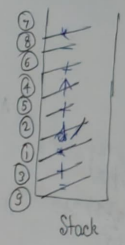
Modifications:

* if precedence = same (of read operator and top)
  (a) if left associative, pop and add, then push
  (b) if right associative push
* if top == '(' : push
* if operator is read and top == '(' : push

e.g.- a = b+c*d/e+f↑g+h-i*j

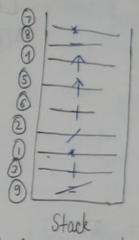
O/P: ~~a~~ bc+de/*

abcd*~~e/~~+fg↑h+ij*--=



e.g.- a = b+c*d/e+f↑g↑h-i*j

O/P: abcd*e/+fgh↑↑+ij*--=



Q) Write an algorithm to convert infix to prefix using stack.

1. Initialize an empty stack
2. Read the infix left to right
3. While (more input):
   (a) if '(' read : push ')' to stack
   (b) if ')' read : push '(' to stack
   (c) else : push the read element to stack

} Reverse the expression and switch '(' and ')'

4. While (stack not empty): pop the stack and add to create a new expression
5. Apply infix to postfix conversion algorithm
6. Read the expression and push each element to stack
7. Pop the stack until stack is empty and add to output.

} Reverse the new expression to get final result

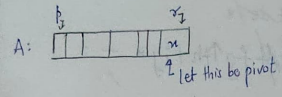
Q) Write an algorithm to check delimiters are balanced or not.

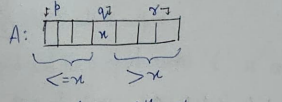
Quicksort Algo

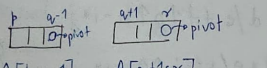
"Divide and Conquer"

QUICKSORT(A, p, r)

1. if (p < r):
   - (a) q = PARTITION(A, p, r)
   - (b) QUICKSORT(A, p, q-1)
   - (c) QUICKSORT(A, q+1, r)





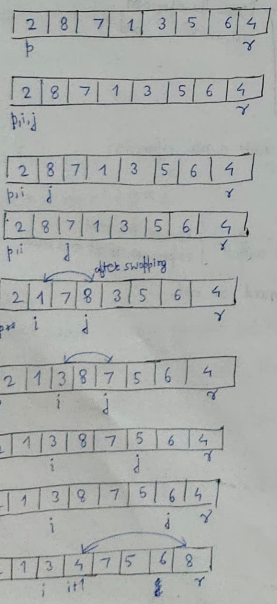


PARTITION(A, p, r)

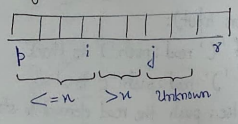
1. x = A[r] // pivot
2. i = p-1
3. for j = p to r-1:
4. if (A[j] <= x):
5. i = i+1
6. Exchange A[j] with A[i]
7. Exchange A[r] with A[i+1]
8. Return i+1

T(n) for PARTITION = O(n)

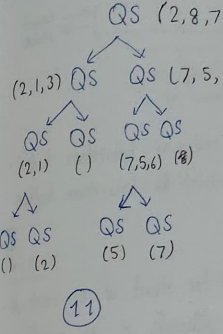
e.g.-



At a given time:

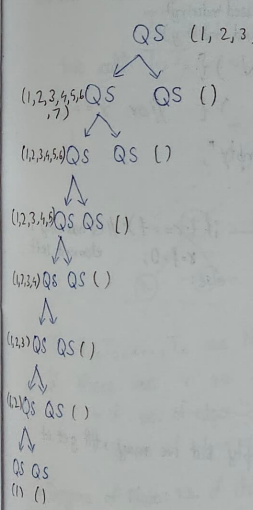


- No. of recursive calls for: 2,8,7,1,3,5,6,4



(11)

- No. of recursive calls for sorted: 1,2,3,4,5,6,7,8



(25)

* This is average case i/p:

→ $O(n \log_2 n)$

Avg Case/Best Case:

$$T(n) = \begin{cases} 2 \cdot T(\frac{n}{2}) + n, & n > 1 \\ 1, & n == 1 \end{cases}$$

$$= 2\left[2 \cdot T(\frac{n}{4}) + \frac{n}{2}\right] + n$$

$$= 2^2\left[T(\frac{n}{4}) + \frac{n}{4}\right] + n$$

; K times

$$= 2^k\left[T(\frac{n}{2^k})\right] + kn \quad (\text{let } \frac{n}{2^k} = 1)$$

$$= 2^k + kn = n + n\log_2 n \approx n\log_2 n$$

* For QuickSort, this is the worst case input. Also, if array is reversely sorted, it is also worst case input (tree will be rightly-skewed)

→ $O(n^2)$

Worst Case:

$$T(n) = \begin{cases} T(n-1) + n, & n > 1 \\ 1, & n == 1 \end{cases}$$

$$= T[T(n-2) + (n-1)] + n$$

$$= T(n-2) + 2n - 1$$

$$= T[T[T(n-3) + (n-3)] + (n-2)] + (n-1) + n$$

; K times

$$= T(n-k) + kn - \sum_{i=0}^{k-1} i \quad (\text{let } n-k = 1)$$

$$= 1 + (n-1)n - \frac{(n-1)(n-1+1)}{2}$$

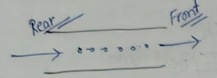
$$\approx n^2$$

* Recurrence Relation – captures asymptotic behaviour of recursive function

- To avoid this, we can choose pivot element randomly each time. This is known as "Randomized Quick Sort".

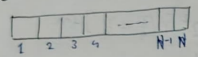
Queue

- Linear List
- FIFO (First-In-First-Out)
  - ↳ - data added from "rear end"
  - data removed from "front end"



- enqueue/enque — adds element to ~~front~~ rear side
- delqueue/deque/delque — deletes element from front side

Array Implementation:



r → points to rear end
f → points to front end

- Initially: r = f = 0 (for 1-based indexing)
  = f = -1 (for 0-based indexing)

```
Insert(Q, r, f, N, x){
    if ( r == N ){
        "Q is Full";
        exit(1);
    }
    r = r+1;
    Q[r] = x;
    if (f == 0)
        f = 1;
}
```

```
Delque(Q, r, f, N ){
    if ( f == 0 ) {   //or r == 0
        "Q is Empty";
        exit(1);
    }
    x = Q[f];          if (r == f): // Q has only 1 element left
    f = f + 1;             r = f = 0;
                       else:
    return x;
}
```

*This is a "linear" or "sequential" queue.

- Limitation: Lack of Space Utilization (as queue can be mostly empty but we may still get it as full.

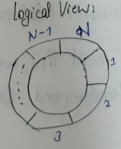
"Circular Queue"

- We don't require circular queue in linked list implementation.

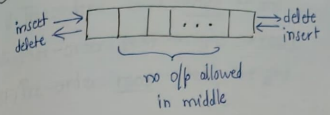
```
Insert(Q, r, f, N, x){
    if ( (r+1 == f  OR  r == N) AND f == 1 ){
        "Q is full";
        exit(1);
    }
    if (r == N)
        r = 1;
    else:
        r = r+1;
    Q[r] = x;
    if (f == 0)
        f = 1;
}
// Cycle back to start
```

```
Delete(Q, r, f, N){
    if ( f == 0 ){
        "Q is empty";
        exit(1);
    }
    x = Q[f]
    ....
    return x;
    if (f == r):
        r = f = 0;
    elseif (f == N): f = 1; // Cycle back to start
    else: f = f+1;
}
```

Logical View:



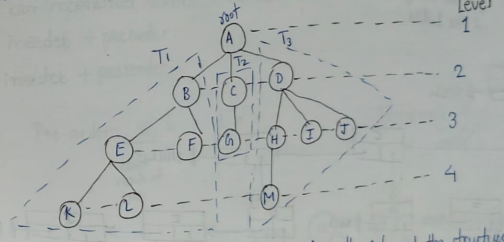
Double-Ended Queue



- input restricted: insertion only allowed at one end
- output restricted: ~~is~~ deletion only allowed at one end

Tree

- A tree is a finite set of one or more nodes such that
  - i) There is a specially designated node called "root" node
  - ii) The remaining nodes are partitioned into $n \geq 0$ disjoint sets:
    $T_1, T_2, \ldots, T_n$, where each of these sets is a tree.
- We call $T_1, T_2, \ldots, T_n$ ~~are~~ subtrees of the root node



- If $T_1, T_2, \ldots, T_n$ are NOT disjoint, a cycle will exist and the structure is no longer a tree
- If there are 'n' no. of nodes, a tree must have 'n-1' edges
  - if no. of edges > n-1 ⇒ cycle exists } NOT trees
  - if no. of edges < n-1 ⇒ isolated nodes

* Degree of Node: no. of children the node has (in a tree)
* Degree of Tree: max^m possible degree of any node in the tree

* Height of Node: distance of that node from the root node
* Depth of Node: distance of that node from its deepest leaf node (highest level)

- Leaf Node: node with degree 0

* Height of Tree: max^m height of any node possible in the tree
* Depth of Tree: max^m depth of any node possible in the tree

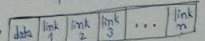
Height of Tree = Depth of Tree

Representation of Tree

- Pictorial Representation
- List Representation
  ↳ A(B(E(K,L),F), C(G), D(H,I,J))

Representation of Tree in memory

- Linked List Representation

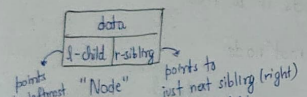


"Node"

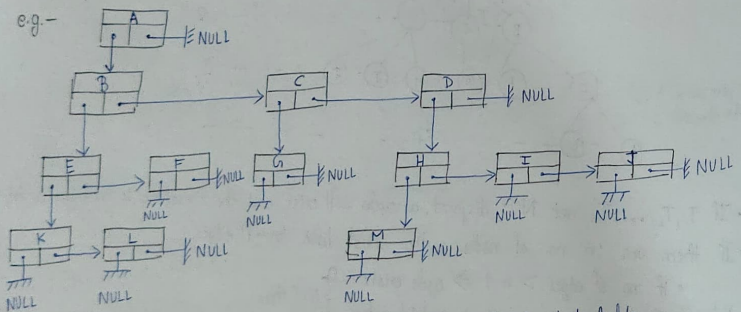
(If we put a constraint that any node can have almost 'n' nodes)

- Here, resources are wasted if only few nodes have the 'n' children.

Left-Child - Right Sibling Representation



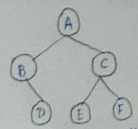
e.g.-



- Here, a node can have any arbitrary no. of children using only two link fields

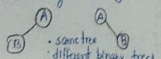
Binary Tree

- Each node can have almost 2 children — left child and right child
- As per def", trees can't be empty but binary trees can be.



In-Order: B D A E C F
Post-Order: D B E F C A
Pre-Order: A B D C E F

- In ordinary trees, there is no ordering among children but in binary tree, we have ordering.





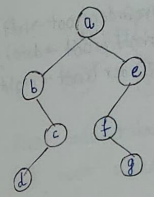
"Node"

```
struct binaryNode{
    int data;
    struct binaryNode* left;
    struct binaryNode* right;
};
```

Binary Tree Traversal

i) In-order: Left → root → right
ii) Post-order: Left → right → root
iii) Pre-order: root → left → right

- Converse In-order: right → root → left
- Converse Post-order: right → left → root
- Converse Pre-order: root → right → left

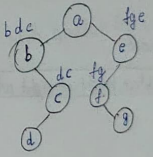


In-order: b d c a f g e
Post-order: d c b g f e a
Pre-order: a b c d e f g

- We can reconstruct a binary tree if we have:
  i) inorder + preorder
  ii) inorder + postorder

Pre-order: a b c d e f g
(a → root node of tree)

Inorder: b d c | a | f g e
(left of a | right of a)



* To check for root node, we check which node comes first in pre-order
* To check for left node, we check which node comes first in inorder

- For a binary tree with 'n' no. of nodes:
  - Max^m height: n
  - Min^m height: $\log_2 n + 1$ (if level starts at 1)
- No. of binary trees possible for 'n' unlabelled nodes:

Q.) Write a procedure to display preorder traversal of a binary tree.

```
struct binaryNode{
    int data;
    struct binaryNode* left;
    struct binaryNode* right;
};
void preOrder(struct BinaryNode* root){
    if (root == NULL)
        return;
    printf("%d", root->data);
    preOrder(root->left);
    preOrder(root->right);
}
```

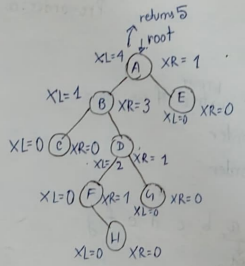
Postorder

```
preOrder(root->left);
preOrder(root->right);
printf("%d", root->data);
```

Inorder

```
preOrder(root->left);
printf("%d", root->data);
preOrder(root->right);
```

Q.)
```
int xyz(struct BTnode* root){
    if (!root)
        return 0;
    int xL = xyz(root->left);
    int xR = xyz(root->right);
    if (xL <= xR)
        return xR+1;
    else
        return xL+1;
}
```



* This returns height of the given node

Height = max (height of left subtree, height of right subtree) + 1

Q.)
```
int getValue(struct BTnode* t){
    int value = 0;
    if(t){
        if(t->left == NULL && t->right == NULL)
            value = 1;
        else
            value += getValue(t->left) + getValue(t->right);
    }
    return value;
}
```

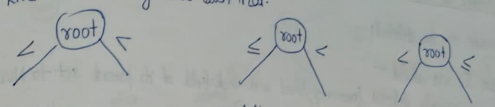
* This returns no. of leaf nodes if root node of a binary tree is passed.

Q.) Why BinaryTree is NOT a tree?

Ans: b/c binarytree can be empty but trees can't be

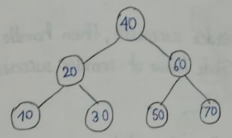
Binary Search Tree

- A special kind of a binary tree such that:



BST | left-biased BST (generally used) | right-biased BST

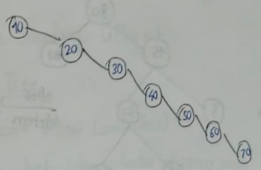
Creation: 40, 20, 30, 60, 50, 70, 10



→ Worst case: $O(\log_2 n)$ (Searching)

- First node taken as root node
- Each node added as a leaf node (maintaining condition of BST)

10, 20, 30, 40, 50, 60, 70



→ Worst Case: O(n) (Searching)

Q.) Write a function to return node with smallest key value in BST.

```
int Smallest(BNode* t){
    if(!t)
        return NULL;
    if(t->left)
        return Smallest(t->left);
    return t->data;
}
```

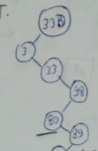
Q.) Identify valid/invalid search sequence for searching 88 in a BST.

(a) 11, 111, 22, 55, 98, 89, 65, 88 → Valid

(b) 333, 3, 33, 98, 80, 89, 35, 88 → Invalid

(c) 9, 99, 19, 29, 96, 82, 89, 85, 88 → Valid

(d) 22, 122, 92, 62, 72, 82, 85, 88 → Valid



(35 is in right subtree of 80 but 35 < 80 so error)

Deleting a Node from a BST

Procedure: A node to be deleted:

Case I: no child —
simply deleted (memory freed and link field of its parent set to NULL)

Case II: only one child (subtree) —
root of its subtree attached with its grandparent through the same link field the parent was.

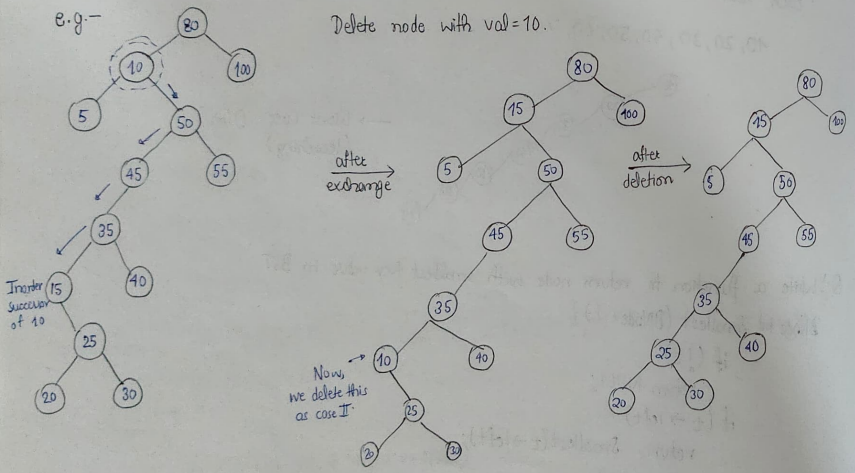
Case III: both subtrees —

(a) replace node to be deleted with its inorder successor, then handle as one of the above cases, for deletion of node (now at inorder successor's position)

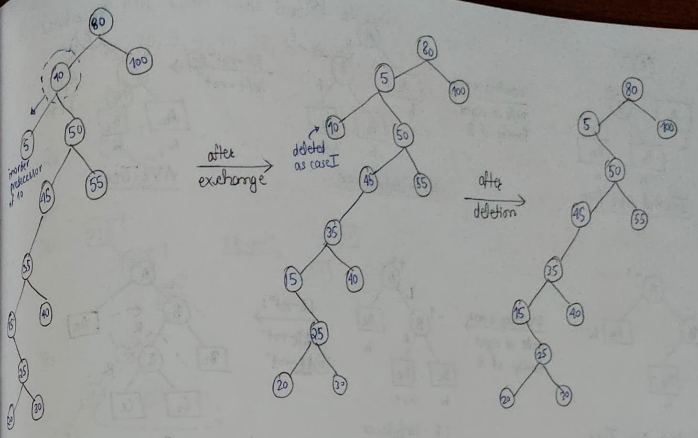
OR

(b) replaced with its inorder predecessor, then handled as case I or II, for deletion of the node (now at inorder predecessor's position)

e.g. —



* For inorder successor, go right once then to the deepest left
* For inorder predecessor, go left once then to the deepest right

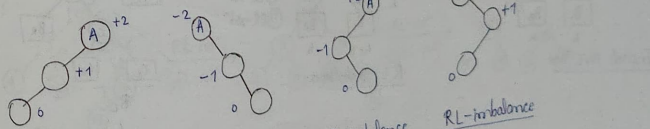


* Balance Factor: (height of left subtree − height of right subtree)
* For balanced BST (i.e height bounded by $\log_2 n$): balance factor $\in \{0, 1, -1\}$
  ↳ Worst case Searching: $O(\log_2 n)$

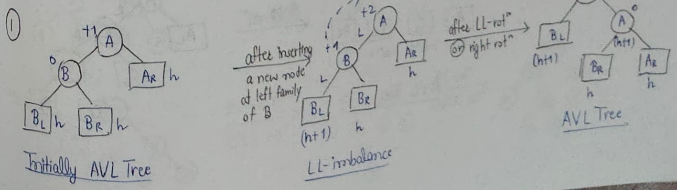
Height Balanced Tree

AVL Tree (Aldeson Velsaci Landis Tree)
↳ a BST in which balance factor of every node is 0, −1 or 1.

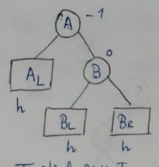
* BF: (height of left subtree − height of right subtree)
* Used to maintain height of tree as $\log_2 n$, so worst case search complexity: $O(\log_2 n)$



LL-imbalance | RR-imbalance | LR-imbalance | RL-imbalance

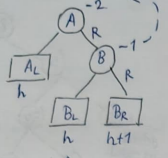


II)



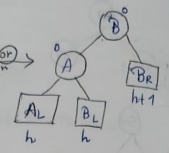
Initial AVL Tree

inserting a new node at right family of B



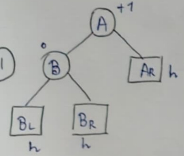
RR-imbalance

RR-rot^n or left-rot^n



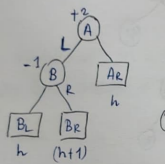
AVL Tree

III)



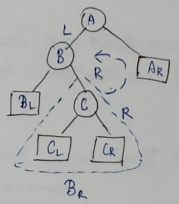
Initial AVL Tree

inserting a new node at right family of B

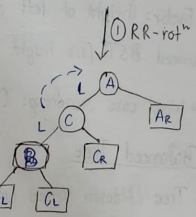


LR-imbalance

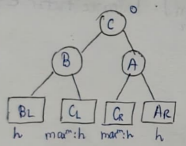
LR-rot^n:
I) RR-rot^n
II) LL-rot^n



I) RR-rot^n

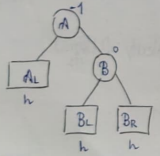


II) LL-rot^n



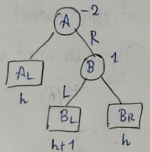
AVL Tree

IV)



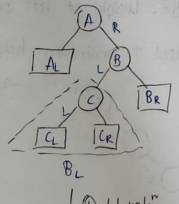
Initial AVL Tree

inserting new node at left family of B

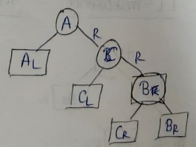


RL-imbalance

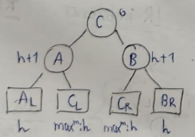
RL-rot^n:
I) LL-rot^n
II) RR-rot^n



I) LL rot^n



II) RR-rot^n



AVL Tree

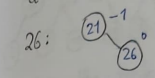
0) Create an AVL with nodes inserted in order:

21, 26, 30, 9, 4, 14, 28, 18, 15, 10, 2, 3, 7

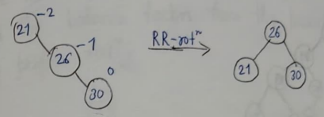
21:



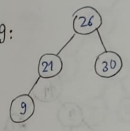
26:



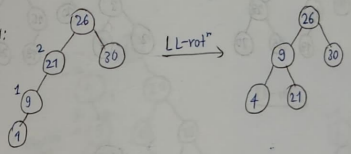
30:



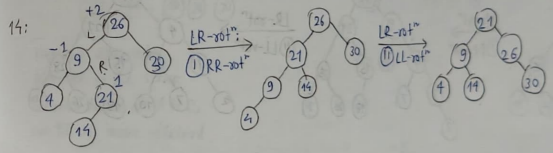
9:



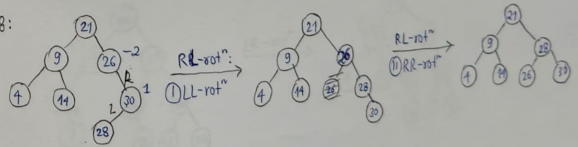
4:



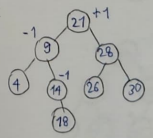
14:



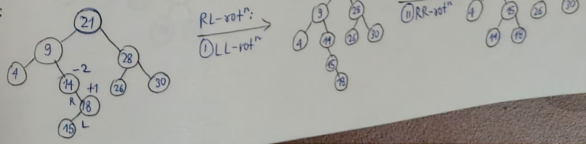
28:



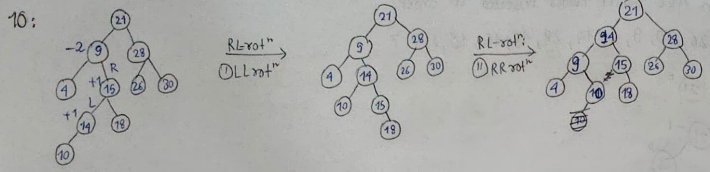
18:



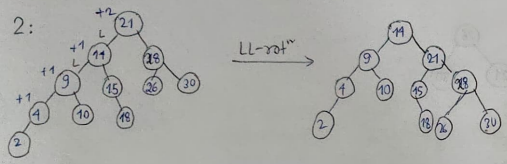
15:



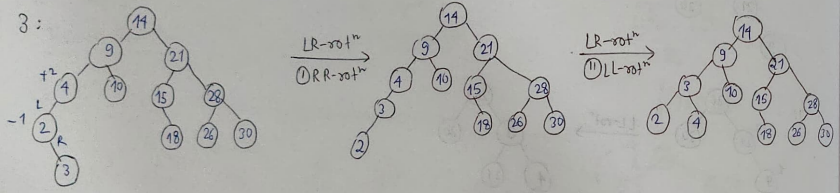
10:



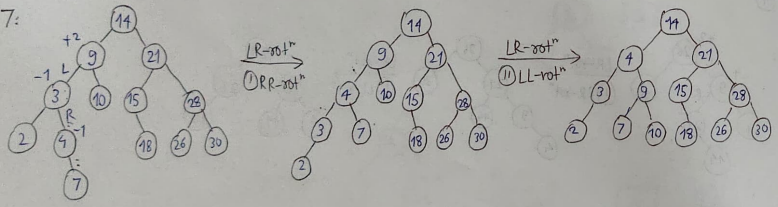
2:



3:

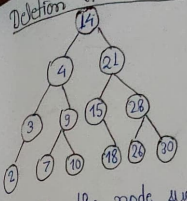


7:



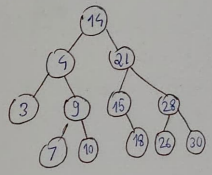
Deletion of nodes from AVL tree

Delete: 2, 3, 10, 18, 4, 9, 14, 7, 15

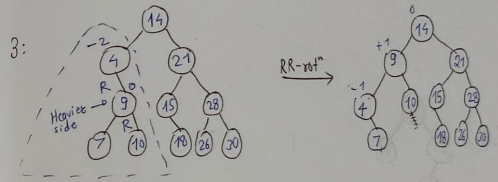


i) delete the node such that from a BST
ii) check the balance factors from the parent of deleted node up till the root node and correct using proper rot^n.

2:

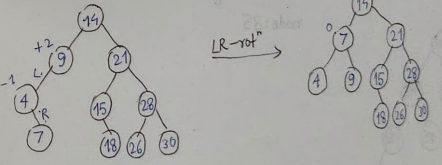


3:

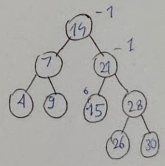


Here, we can take both, RR or RL since balance factor of 9 is 0, but RR is more efficient

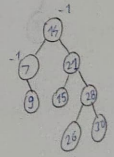
10:



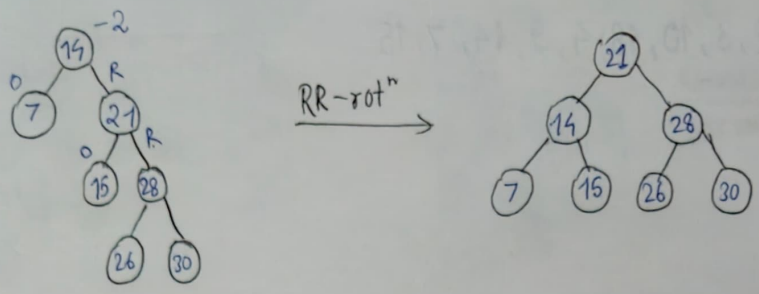
18:



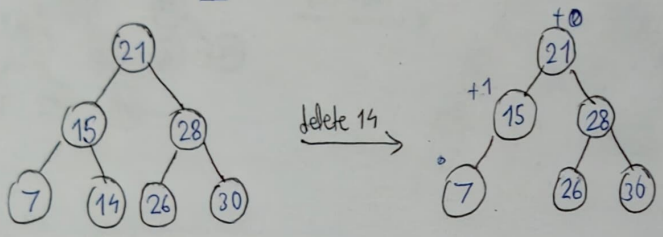
4:



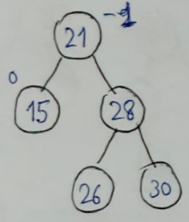
9:



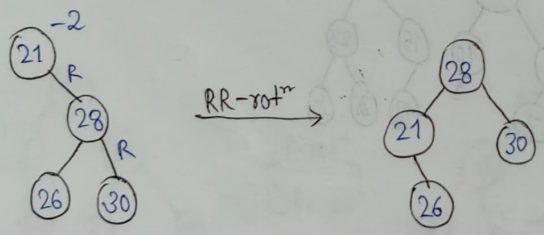
14: Inorder: 7, 14, 15, 21, 26, 28, 30



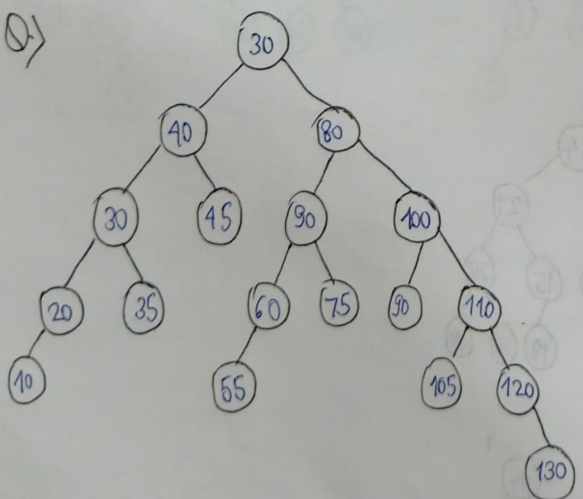
7:



15:



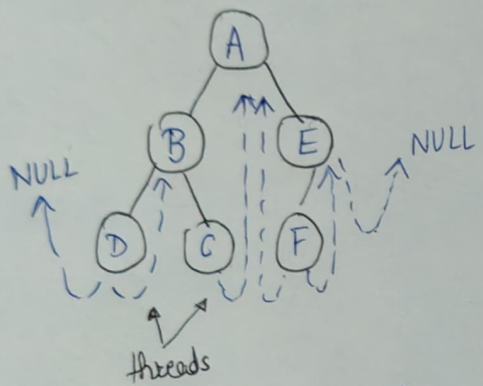
Q)



Show resulting AVL tree after deleting node: 35

Threaded Binary Tree

- If a node doesn't have a left or right child, instead of pointing to NULL, left child points to inorder predecessor and right child points to inorder successor (if they exist)



- We maintain flag variables for when nodes are pointing to inorder successors / predecessors instead of right / left childs.
- If inorder predecessor / successor don't exist, they can point to NULL or head.
- This is used for efficient traversal.